TSA Forecasting_report_top5

Bruce Chen, Jeffery Chu

2026-04-23

> Summary of results: The TBATS model achieved the best performance due to its ability to capture multiple seasonal patterns in daily electricity demand. Other models incorporating exogenous variables did not significantly improve accuracy. Model performance rank from best to worst:
>
> 1. TBATS
> 2. SARIMAX
> 3. STLM
> 4. MSTL_ETS
> 5. ARIMA + Fourier
>
> Github links: https://github.com/Parautilitarian/TSA_Load_Forecasting (https://github.com/Parautilitarian/TSA_Load_Forecasting)

```
library(lubridate)
```

```
##
## Attaching package: 'lubridate'
```

```
## The following objects are masked from 'package:base':
##
##     date, intersect, setdiff, union
```

```
library(ggplot2)
library(forecast)
```

```
## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo
```

```
library(Kendall)
library(tseries)
library(outliers)
library(tidyverse)
```

```
## ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
## ✔ dplyr   1.1.4     ✔ stringr 1.5.1
## ✔ forcats 1.0.0     ✔ tibble  3.3.0
## ✔ purrr   1.2.2     ✔ tidyr   1.3.1
## ✔ readr   2.1.5
```

```
## ── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()    masks stats::lag()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts
to become errors
```

```
library(readxl)
library(forecast)
```

1. Load Dataset

Preprocessing

```
load <- read_excel("load.xlsx")
head(load)
```

```
## # A tibble: 6 × 26
##   meter_id date                  h1    h2    h3    h4    h5    h6    h7    h8
##   <chr>    <dttm>             <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 0072     2005-01-01 00:00:00  3072  2911  2784  2768  2832  2904  3064  3192
## 2 0072     2005-01-02 00:00:00  2408  2328  2311  2328  2400  2552  2856  3272
## 3 0072     2005-01-03 00:00:00  2575  2528  2496  2608  2825  3376  3760  3672
## 4 0072     2005-01-04 00:00:00  2177  2016  1951  1913  2080  2624  3328  2896
## 5 0072     2005-01-05 00:00:00  1912  1768  1697  1720  1880  2472  3208  2888
## 6 0072     2005-01-06 00:00:00  2104  1976  1920  1952  2167  2736  3432  3096
## # ℹ 16 more variables: h9 <dbl>, h10 <dbl>, h11 <dbl>, h12 <dbl>, h13 <dbl>,
## #   h14 <dbl>, h15 <dbl>, h16 <dbl>, h17 <dbl>, h18 <dbl>, h19 <dbl>,
## #   h20 <dbl>, h21 <dbl>, h22 <dbl>, h23 <dbl>, h24 <dbl>
```

Convert Hourly to Daily

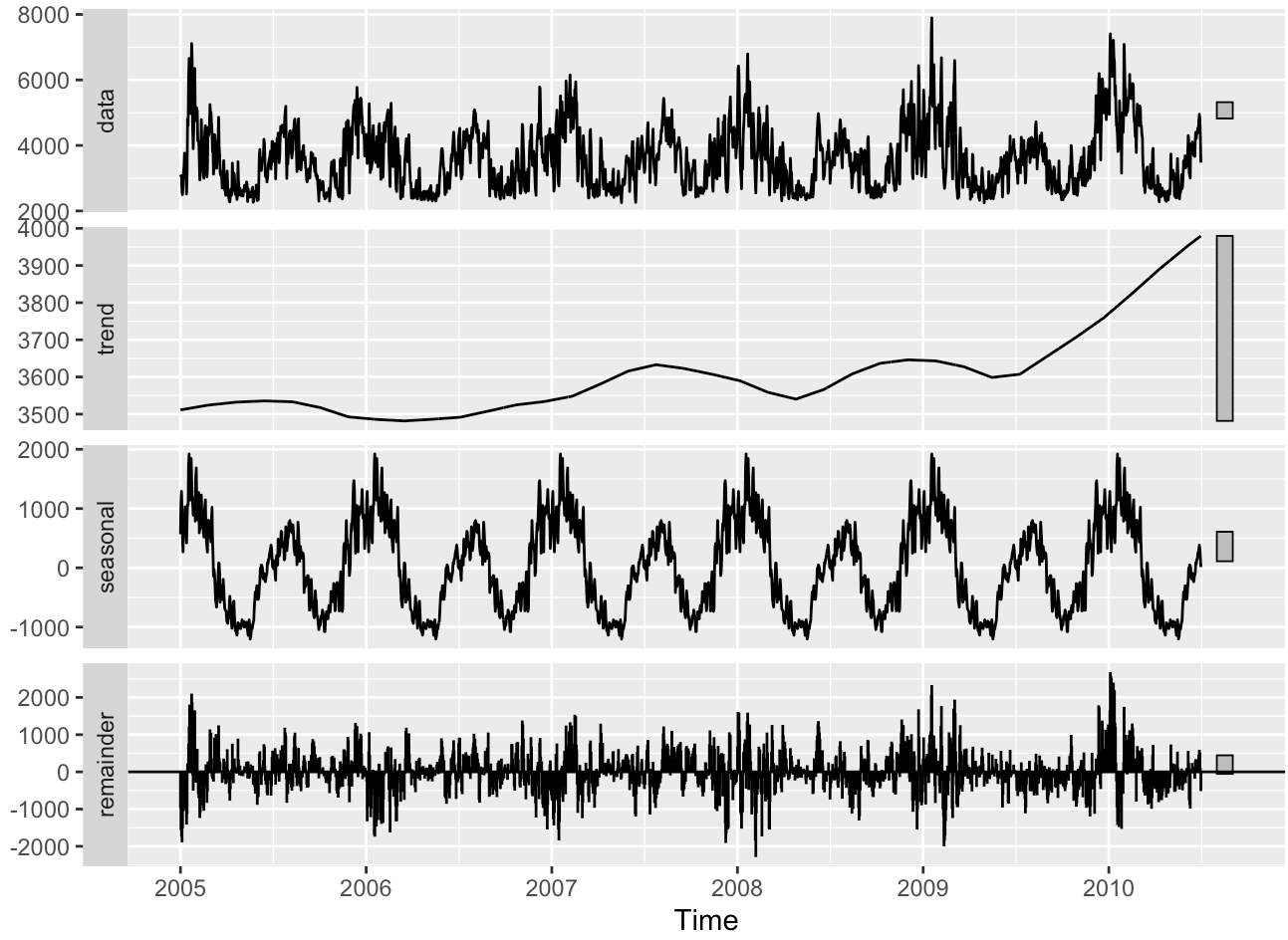
```
daily_load <- load %>%
  rowwise() %>%
  mutate(daily_load = mean(c_across(starts_with("h")), na.rm = TRUE)) %>%
  ungroup() %>%
  select(date, daily_load)

daily_load$date <- as.Date(daily_load$date)
```

Train-Validation Split

```
train_data <- daily_load %>%
  filter(date <= as.Date("2010-06-30"))

valid_data <- daily_load %>%
  filter(date >= as.Date("2010-07-01") &
         date <= as.Date("2010-07-31"))

train_msts <- msts(
  train_data$daily_load,
  seasonal.periods = c(7, 365),
  start = c(2005, 1)
)

train_ts <- ts(train_data$daily_load,
               start = c(2005,1),
               frequency = 365)

valid_msts <- msts(
  valid_data$daily_load,
  seasonal.periods = c(7, 365),
  start = c(2010, 182)
)

valid_ts <- ts(valid_data$daily_load,
               start = c(2010,7),
               frequency = 365)
```

Decomposition

```
decomp <- stl(train_ts, s.window = "periodic")
autoplot(decomp) # seasonal
```



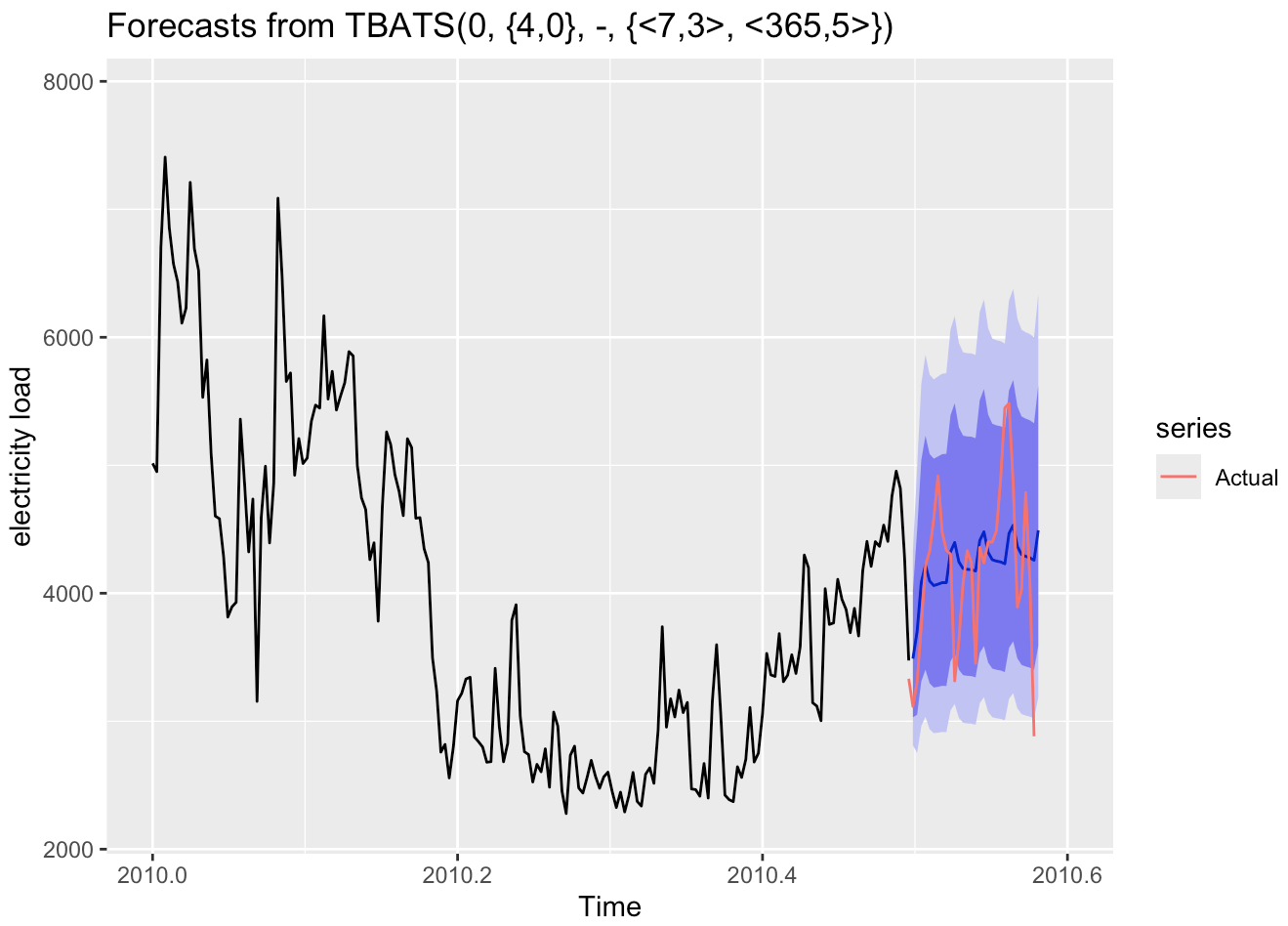
2. Model 1 - TBATS (Baseline Model)

```
fit_tbats <- tbats(train_msts)
forecast_tbats <- forecast(fit_tbats, h = length(valid_msts))

autoplot(forecast_tbats) +
  autolayer(valid_msts, series = "Actual", PI = FALSE) +
  xlim(2010, 2010.6) +
  ylab("electricity load")
```

```
## Warning in ggplot2::geom_line(ggplot2::aes(x = .data[["timeVal"]], y =
## .data[["seriesVal"]], : Ignoring unknown parameters: `PI`
```

```
## Scale for x is already present.
## Adding another scale for x, which will replace the existing scale.
```

```
## Warning: Removed 1825 rows containing missing values or values outside the scale rang
e
## (`geom_line()`).
```




```
summary(fit_tbats)
```

```
##                    Length Class  Mode
## lambda                 1  -none- numeric
## alpha                  1  -none- numeric
## beta                   0  -none- NULL
## damping.parameter      0  -none- NULL
## gamma.one.values       2  -none- numeric
## gamma.two.values       2  -none- numeric
## ar.coefficients        4  -none- numeric
## ma.coefficients        0  -none- NULL
## likelihood             1  -none- numeric
## optim.return.code      1  -none- numeric
## variance               1  -none- numeric
## AIC                    1  -none- numeric
## parameters             2  -none- list
## seed.states           21  -none- numeric
## fitted.values       2007  msts   numeric
## errors              2007  msts   numeric
## x                  42147  -none- numeric
## seasonal.periods       2  -none- numeric
## k.vector               2  -none- numeric
## y                   2007  msts   numeric
## p                      1  -none- numeric
## q                      1  -none- numeric
## call                   2  -none- call
## series                 1  -none- character
## method                 1  -none- character
```

```
accuracy(forecast_tbats, valid_msts)
```

```
##                     ME     RMSE      MAE        MPE      MAPE      MASE
## Training set 29.98179 423.5522 313.6325 -0.3333858  8.518809 0.4442748
## Test set     11.41074 561.2408 429.5423 -1.6755046 10.665483 0.6084665
##                     ACF1 Theil's U
## Training set 0.02262869        NA
## Test set     0.48200168  1.030122
```

3. Temperature Dataset

Preprocessing

```
# load data
temperature <- read_excel("temperature.xlsx")
head(temperature)
```

```
## # A tibble: 6 × 30
##   date                  hr t_ws1 t_ws2 t_ws3 t_ws4 t_ws5 t_ws6 t_ws7 t_ws8
##   <dttm>              <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 2005-01-01 00:00:00     1    43    46    40    47    48    46    44    52
## 2 2005-01-01 00:00:00     2    41    46    38    46    48    45    51    50
## 3 2005-01-01 00:00:00     3    40    46    37    45    45    45    49    48
## 4 2005-01-01 00:00:00     4    39    46    37    47    48    48    45    50
## 5 2005-01-01 00:00:00     5    38    46    37    44    48    49    43    50
## 6 2005-01-01 00:00:00     6    37    45    36    45    48    48    40    50
## # ℹ 20 more variables: t_ws9 <dbl>, t_ws10 <dbl>, t_ws11 <dbl>, t_ws12 <dbl>,
## #   t_ws13 <dbl>, t_ws14 <dbl>, t_ws15 <dbl>, t_ws16 <dbl>, t_ws17 <dbl>,
## #   t_ws18 <dbl>, t_ws19 <dbl>, t_ws20 <dbl>, t_ws21 <dbl>, t_ws22 <dbl>,
## #   t_ws23 <dbl>, t_ws24 <dbl>, t_ws25 <dbl>, t_ws26 <dbl>, t_ws27 <dbl>,
## #   t_ws28 <dbl>
```

```
# convert hourly to daily
temp_daily <- temperature %>%
  mutate(date = as.Date(date, format="%d%b%Y")) %>%
  group_by(date) %>%
  summarise(across(-hr, mean, na.rm = TRUE))
```

```
## Warning: There was 1 warning in `summarise()`.
## ℹ In argument: `across(-hr, mean, na.rm = TRUE)`.
## ℹ In group 1: `date = 2005-01-01`.
## Caused by warning:
## ! The `...` argument of `across()` is deprecated as of dplyr 1.1.0.
## Supply arguments directly to `.fns` through an anonymous function instead.
##
##   # Previously
##   across(a:b, mean, na.rm = TRUE)
##
##   # Now
##   across(a:b, \(x) mean(x, na.rm = TRUE))
```

```
# split train data
temp_train <- temp_daily %>%
  filter(date <= as.Date("2010-06-30"))

# convert to time series
temp_train_ts <- ts(
  temp_train[,-1],
  start = c(2005,1),
  frequency = 365
)
```

Determining each station's level of correlations with load

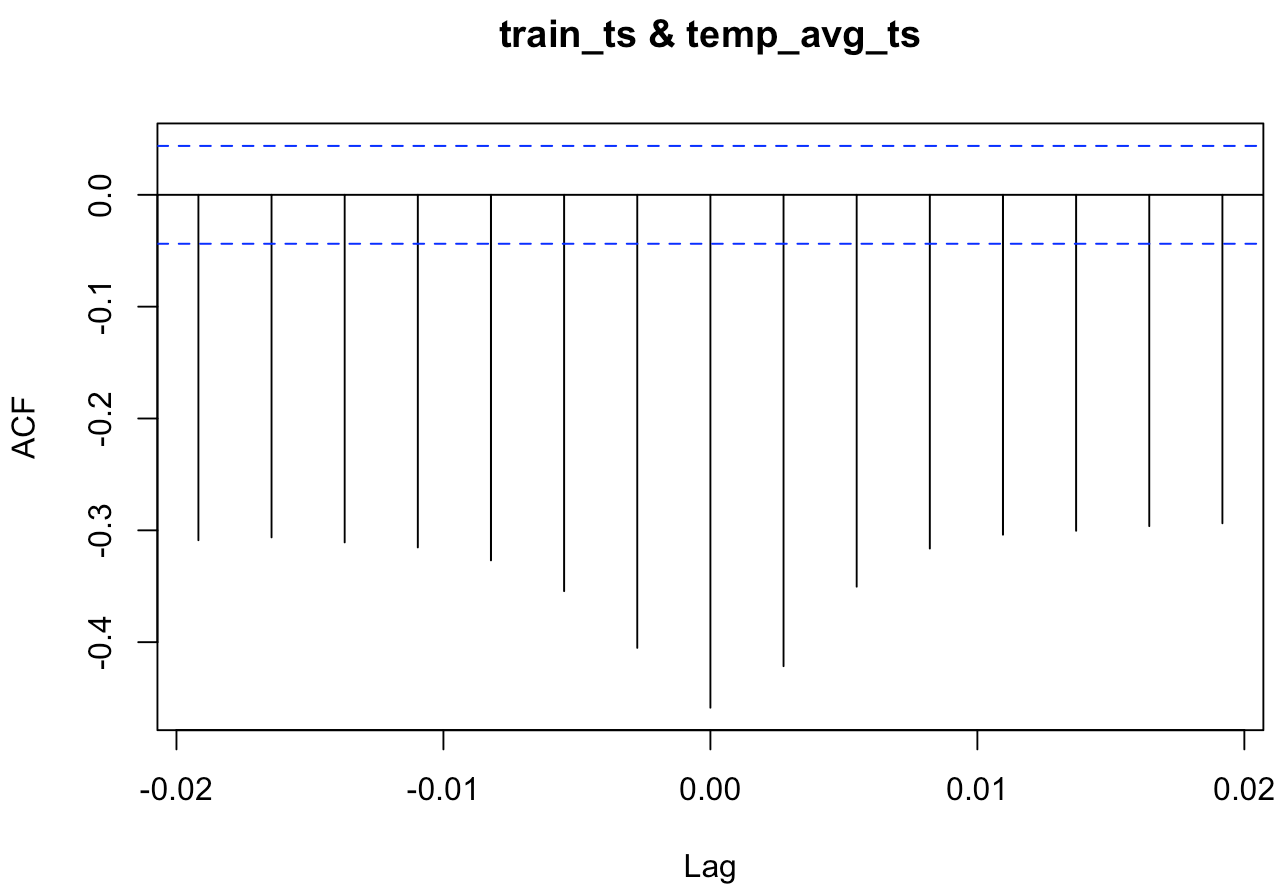
```
cors <- cor(train_ts, temp_train_ts, use="complete.obs")
sort(cors, decreasing = TRUE)
```

```
##  [1] -0.4136529 -0.4187522 -0.4362185 -0.4365159 -0.4408861 -0.4442132
##  [7] -0.4447486 -0.4498241 -0.4505035 -0.4506287 -0.4519990 -0.4528129
## [13] -0.4545288 -0.4559035 -0.4571899 -0.4572746 -0.4576779 -0.4594995
## [19] -0.4598152 -0.4603253 -0.4612840 -0.4613919 -0.4626571 -0.4635158
## [25] -0.4641586 -0.4670773 -0.4680701 -0.4690951
```

- **All correlations are around -0.41 to -0.47. That means:**
  - **Temperature and load are moderately negatively correlated.**
  - **The stations behave very similarly (only ~0.05 difference between best and worst).**

```
# average all station data since their correlations are similar
temp_avg_ts <- ts(
  rowMeans(temp_train_ts, na.rm = TRUE),
  start = start(temp_train_ts),
  frequency = frequency(temp_train_ts)
)
```

```
ccf(train_ts, temp_avg_ts, lag.max = 7)
```



Investigating correlations between average temperature and a lagged load series showed no improvement in correlations.

4. Humidity Dataset

Preprocessing

```
# load data
humidity <- read_excel("relative_humidity.xlsx")
head(humidity)
```

```
## # A tibble: 6 × 30
##   date                   hr rh_ws1 rh_ws2 rh_ws3 rh_ws4 rh_ws5 rh_ws6 rh_ws7
##   <dttm>              <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
## 1 2005-01-01 00:00:00     1     99     93     93     90     87     93     93
## 2 2005-01-01 00:00:00     2     76     93     97     89     87     97     80
## 3 2005-01-01 00:00:00     3     79     93     93     90     93     97     83
## 4 2005-01-01 00:00:00     4     79     93     93     90     87     89     90
## 5 2005-01-01 00:00:00     5     79     93     96     93     87     90     93
## 6 2005-01-01 00:00:00     6     82     93     97     97     87     93     97
## # ℹ 21 more variables: rh_ws8 <dbl>, rh_ws9 <dbl>, rh_ws10 <dbl>,
## #   rh_ws11 <dbl>, rh_ws12 <dbl>, rh_ws13 <dbl>, rh_ws14 <dbl>, rh_ws15 <dbl>,
## #   rh_ws16 <dbl>, rh_ws17 <dbl>, rh_ws18 <dbl>, rh_ws19 <dbl>, rh_ws20 <dbl>,
## #   rh_ws21 <dbl>, rh_ws22 <dbl>, rh_ws23 <dbl>, rh_ws24 <dbl>, rh_ws25 <dbl>,
## #   rh_ws26 <dbl>, rh_ws27 <dbl>, rh_ws28 <dbl>
```

```
# convert hourly to daily
humi_daily <- humidity %>%
  mutate(date = as.Date(date, format="%d%b%Y")) %>%
  group_by(date) %>%
  summarise(across(-hr, mean, na.rm = TRUE))

# split train data
humi_train <- humi_daily %>%
  filter(date <= as.Date("2010-06-30"))

# convert to time series
humi_train_ts <- ts(
  humi_train[,-1],
  start = c(2005,1),
  frequency = 365
)
```

Determining each station's level of correlations with load

```
cors_humi <- cor(train_ts, humi_train_ts, use="complete.obs")
sort(cors_humi, decreasing = TRUE)
```

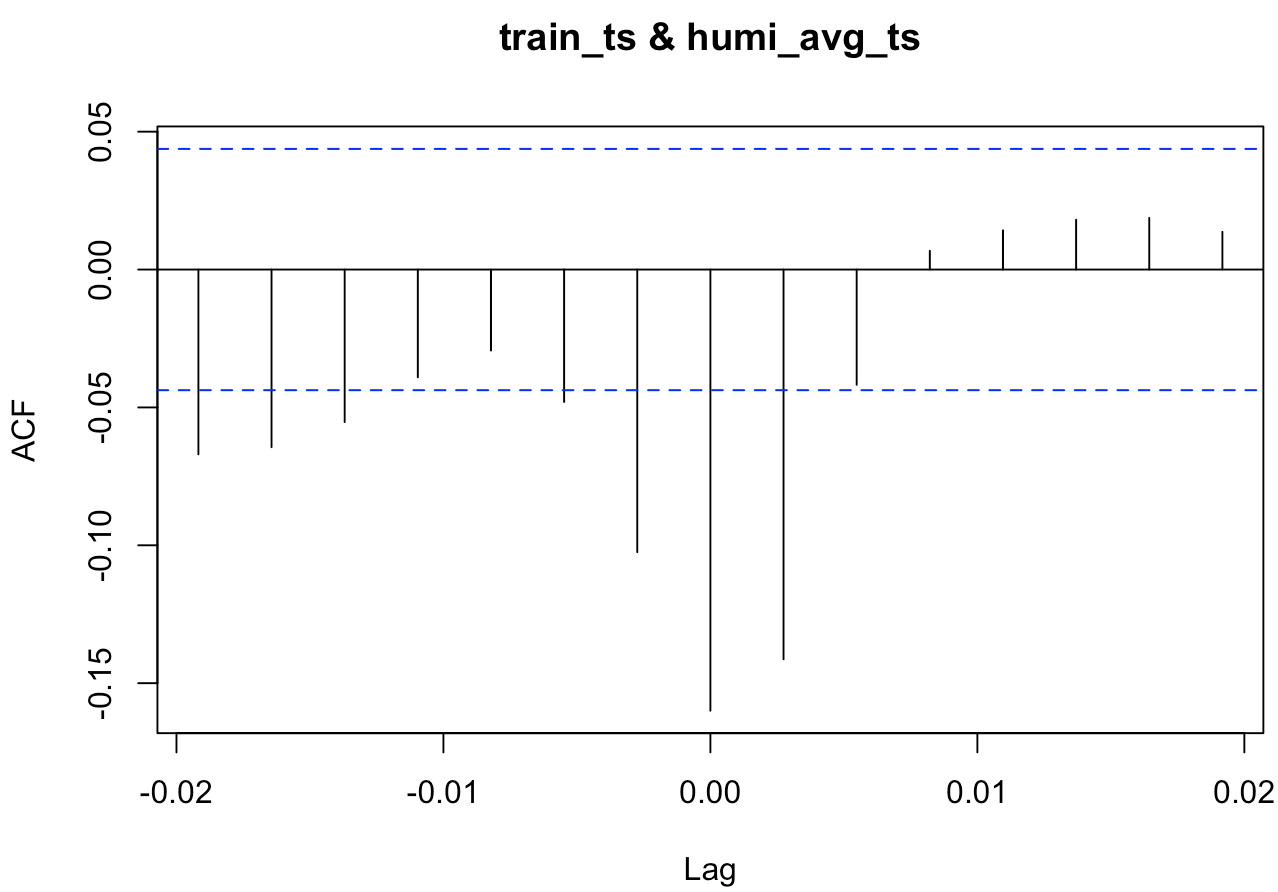
```
##  [1] -0.1099332 -0.1146505 -0.1240858 -0.1369922 -0.1412879 -0.1486531
##  [7] -0.1507341 -0.1645093 -0.1693156 -0.1718564 -0.1754950 -0.1757020
## [13] -0.1811398 -0.1824455 -0.1845058 -0.1895181 -0.1906106 -0.1938069
## [19] -0.2017693 -0.2026199 -0.2189510 -0.2190670 -0.2215253 -0.2262413
## [25] -0.2263705 -0.2270050 -0.2496091 -0.2950316
```

Relative humidity have an even weaker negative correlations with load than the temperature measurements.

I'll average the top 5 stations with a correlation of 0.225 and larger in absolute value.

```
top5 <- order(abs(cors), decreasing = TRUE)[1:5]

humi_avg_ts <- ts(
  rowMeans(humi_train_ts[, top5], na.rm = TRUE),
  start = start(humi_train_ts),
  frequency = frequency(humi_train_ts)
)
```

```
ccf(train_ts, humi_avg_ts, lag.max = 7)
```



Investigating correlations between average temperature and a lagged load series showed no improvement in correlations.

5. Model 2 - SARIMAX Model with Temperature and Humidity

```
ts_train <- ts(
  train_data$daily_load,
  start = c(2005, 1),
  frequency = 365
)

valid_ts <- ts(
  valid_data$daily_load,
  start = c(2010, 182),
  frequency = 365
)

xreg_train <- cbind(
  temp = train_data$temp,
  humi = train_data$humidity
)
```

```
## Warning: Unknown or uninitialised column: `temp`.
```

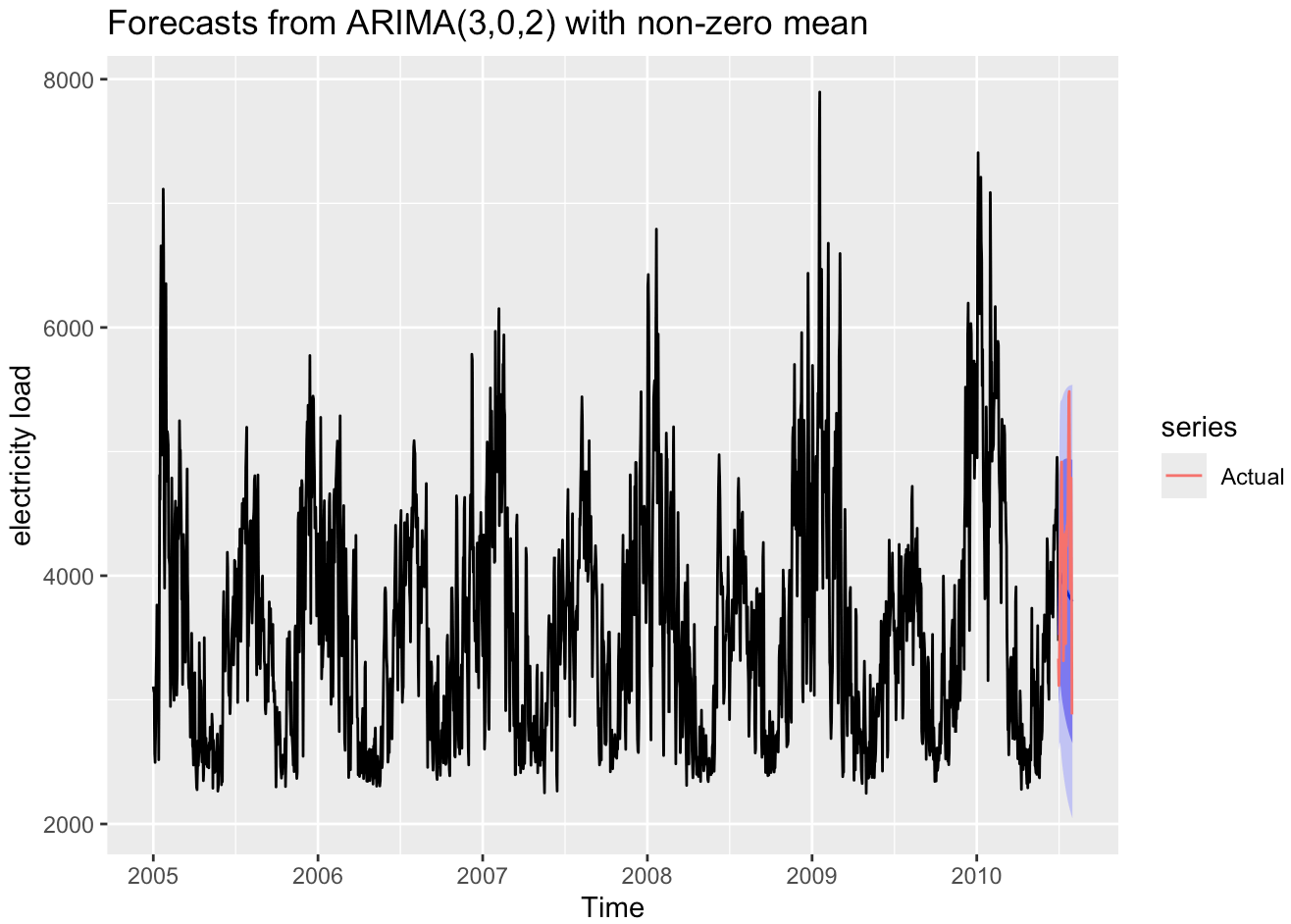
```
## Warning: Unknown or uninitialised column: `humidity`.
```

```
xreg_valid <- cbind(
  temp = valid_data$temp,
  humi = valid_data$humidity
)
```

```
## Warning: Unknown or uninitialised column: `temp`.
## Unknown or uninitialised column: `humidity`.
```

```
fit_sarimax <- auto.arima(
  ts_train,
  xreg = xreg_train,
  seasonal = TRUE
)
```

```
fc_sarimax <- forecast(
  fit_sarimax,
  xreg = xreg_valid,
  h = nrow(valid_data)
)

autoplot(fc_sarimax) +
  autolayer(valid_ts, series = "Actual") +
  ylab("electricity load")
```




```
summary(fit_sarimax)
```

```
## Series: ts_train
## ARIMA(3,0,2) with non-zero mean
##
## Coefficients:
##          ar1      ar2     ar3      ma1      ma2      mean
##       1.6549  -0.8789  0.2097  -0.6649  -0.1123  3596.348
## s.e.  0.0746   0.1295  0.0609   0.0759   0.0718   148.941
##
## sigma^2 = 189278:  log likelihood = -15039.22
## AIC=30092.45   AICc=30092.5   BIC=30131.68
##
## Training set error measures:
##                    ME     RMSE     MAE       MPE     MAPE      MASE
## Training set 0.779334 434.4105 325.745 -1.317964 9.031731 0.4614328
##                      ACF1
## Training set 0.0003196053
```

```
accuracy(fc_sarimax, valid_ts)
```

```
##                      ME     RMSE      MAE       MPE      MAPE      MASE
## Training set   0.779334 434.4105 325.7450 -1.317964  9.031731 0.4614328
## Test set     342.302325 685.4309 568.6735  6.134951 13.156322 0.8055522
##                      ACF1 Theil's U
## Training set 0.0003196053        NA
## Test set     0.5282838277  1.199974
```

6. Model 3 - ARIMA + (Tuned) Fourier Terms

```
best_aicc <- Inf
best_fit <- NULL
best_K <- NULL

for(k1 in 1:3){        # weekly (max 3)
  for(k2 in 1:10){     # yearly (safe range)

    xreg <- fourier(train_msts, K = c(k1, k2))

    fit <- auto.arima(
      train_msts,
      xreg = xreg,
      seasonal = FALSE
    )

    if(fit$aicc < best_aicc){
      best_aicc <- fit$aicc
      best_fit <- fit
      best_K <- c(k1, k2)
    }
  }
}

best_K
```

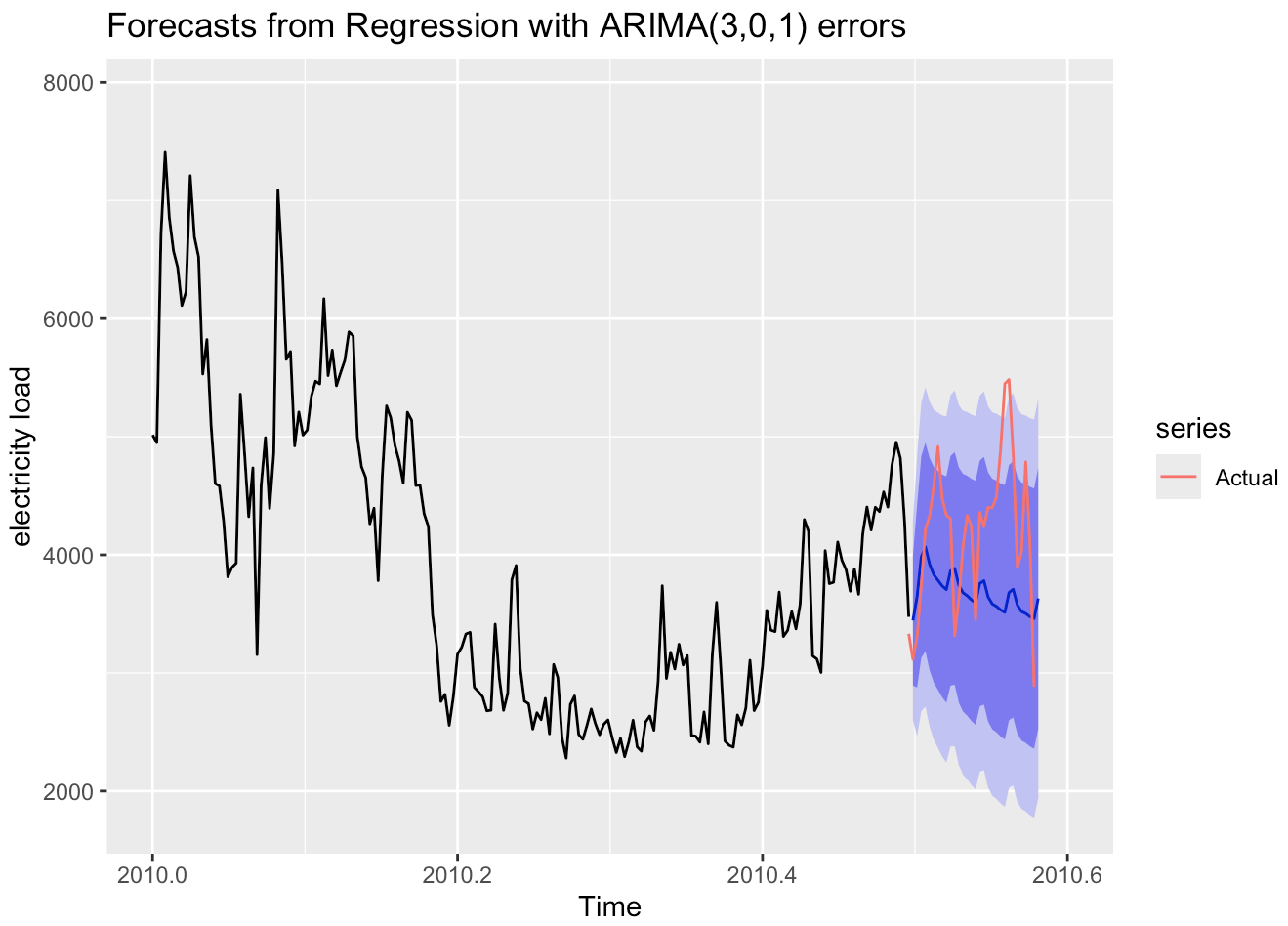
```
## [1] 3 1
```

```
K <- c(3,1)
# 3 Fourier pairs for weekly, 5 for yearly

fourier_train <- fourier(train_msts, K = K)

fit_arima_fourier <- auto.arima(
  train_msts,
  xreg = fourier_train,
  seasonal = FALSE
)

fourier_future <- fourier(train_msts, K = K, h = length(valid_msts))

forecast_arima_fourier <- forecast(
  fit_arima_fourier,
  xreg = fourier_future,
  h = length(valid_msts)
)

autoplot(forecast_arima_fourier) +
  autolayer(valid_msts, series = "Actual", PI = FALSE) +
  xlim(2010, 2010.6) +
  ylab("electricity load")
```

```
## Warning in ggplot2::geom_line(ggplot2::aes(x = .data[["timeVal"]], y =
## .data[["seriesVal"]], : Ignoring unknown parameters: `PI`
```

```
## Scale for x is already present.
## Adding another scale for x, which will replace the existing scale.
```

```
## Warning: Removed 1825 rows containing missing values or values outside the scale rang
e
## (`geom_line()`).
```




```
summary(fit_arima_fourier)
```

```
## Series: train_msts
## Regression with ARIMA(3,0,1) errors
##
## Coefficients:
##          ar1      ar2     ar3      ma1  intercept     S1-7      C1-7     S2-7
##       1.7494  -1.0523  0.2850  -0.7689  3598.0185  93.8291  -14.8674  12.9280
## s.e.  0.0405   0.0457  0.0219   0.0385   122.9653  22.8471   22.8477   9.6095
##           C2-7     S3-7      C3-7    S1-365    C1-365
##       -51.8395  -7.0030  -12.8753  -44.9950  415.4514
## s.e.    9.6084   6.1638    6.1702  158.7735  155.3267
##
## sigma^2 = 184660:  log likelihood = -15010.85
## AIC=30049.7   AICc=30049.91   BIC=30128.16
##
## Training set error measures:
##                    ME     RMSE      MAE       MPE    MAPE      MASE        ACF1
## Training set 1.411585 428.3264 321.6325 -1.258247 8.89326 0.4556072 0.006937466
```

```
accuracy(forecast_arima_fourier, valid_msts)
```

```
##                      ME     RMSE      MAE       MPE     MAPE      MASE
## Training set   1.411585 428.3264 321.6325 -1.258247  8.89326 0.4556072
## Test set     538.940400 824.0364 693.1179 10.849698 15.63340 0.9818334
##                     ACF1 Theil's U
## Training set 0.006937466        NA
## Test set     0.584202933  1.445234
```

8. Model 4 - MSTL + ETS

```
fit_mstl <- mstl(train_msts)

fit_ets <- ets(seasadj(fit_mstl))
```

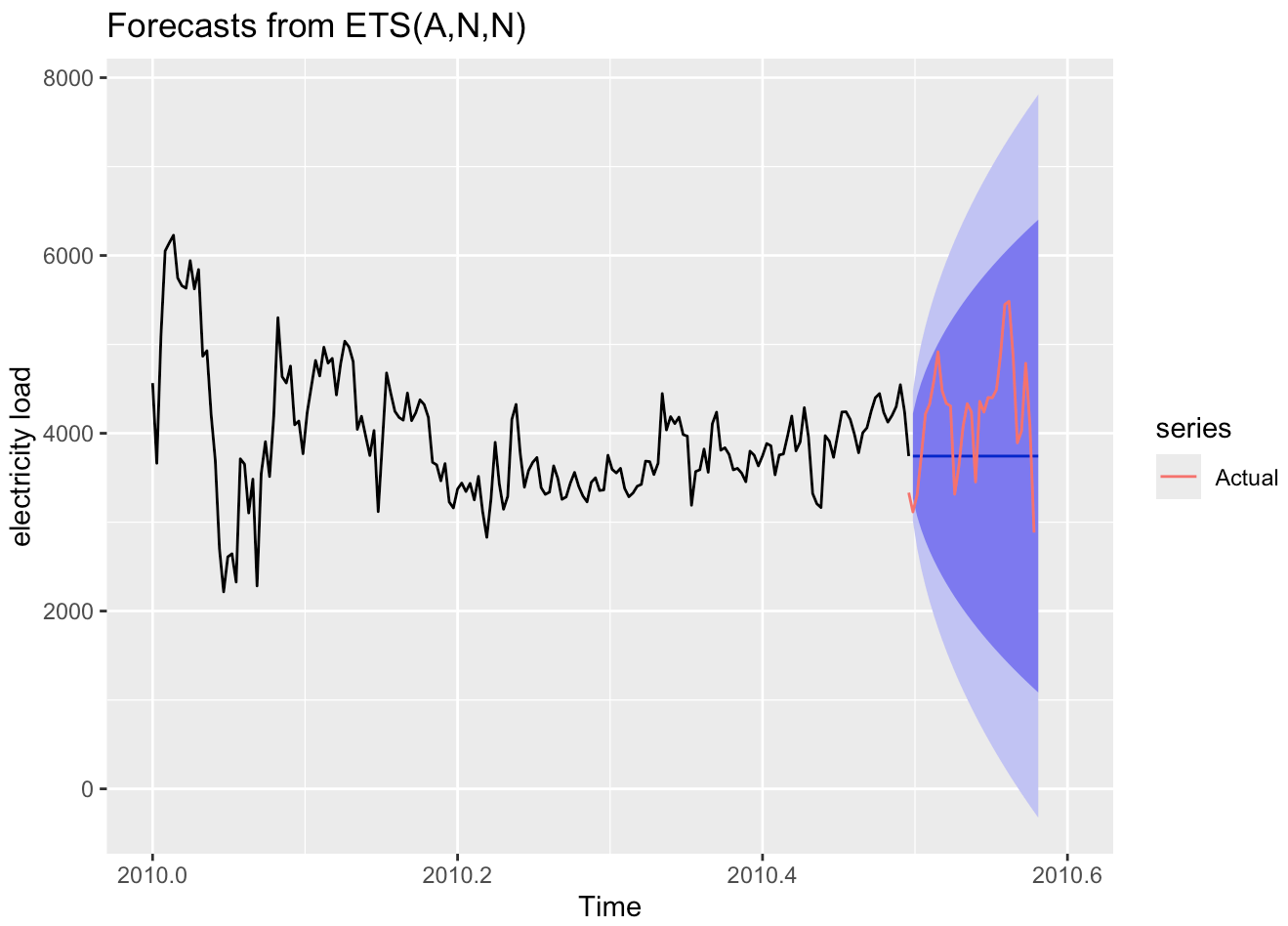
```
## Warning in ets(seasadj(fit_mstl)): I can't handle data with frequency greater
## than 24. Seasonality will be ignored. Try stlf() if you need seasonal
## forecasts.
```

```
fc_mstl <- forecast(fit_ets, h = length(valid_msts))

autoplot(fc_mstl) +
  autolayer(valid_msts, series = "Actual", PI = FALSE) +
  xlim(2010, 2010.6) +
  ylab("electricity load")
```

```
## Warning in ggplot2::geom_line(ggplot2::aes(x = .data[["timeVal"]], y =
## .data[["seriesVal"]], : Ignoring unknown parameters: `PI`
```

```
## Scale for x is already present.
## Adding another scale for x, which will replace the existing scale.
```

```
## Warning: Removed 1825 rows containing missing values or values outside the scale rang
e
## (`geom_line()`).
```




```
summary(fit_ets)
```

```
## ETS(A,N,N)
##
## Call:
## ets(y = seasadj(fit_mstl))
##
##   Smoothing parameters:
##     alpha = 0.9999
##
##   Initial states:
##     l = 2426.1522
##
##   sigma:  372.6854
##
##      AIC     AICc      BIC
## 39031.85 39031.86 39048.66
##
## Training set error measures:
##                     ME     RMSE      MAE        MPE     MAPE      MASE
## Training set 0.6562383 372.4997 274.6701 -0.5485673 7.833149 0.4084209
##                    ACF1
## Training set 0.08022665
```

```
accuracy(fc_mstl, valid_msts)
```

```
##                       ME     RMSE      MAE        MPE      MAPE      MASE
## Training set   0.6562383 372.4997 274.6701 -0.5485673  7.833149 0.4084209
## Test set     477.7163915 775.2139 661.6003  9.2966415 15.143830 0.9837669
##                    ACF1 Theil's U
## Training set 0.08022665        NA
## Test set     0.50591914  1.353303
```

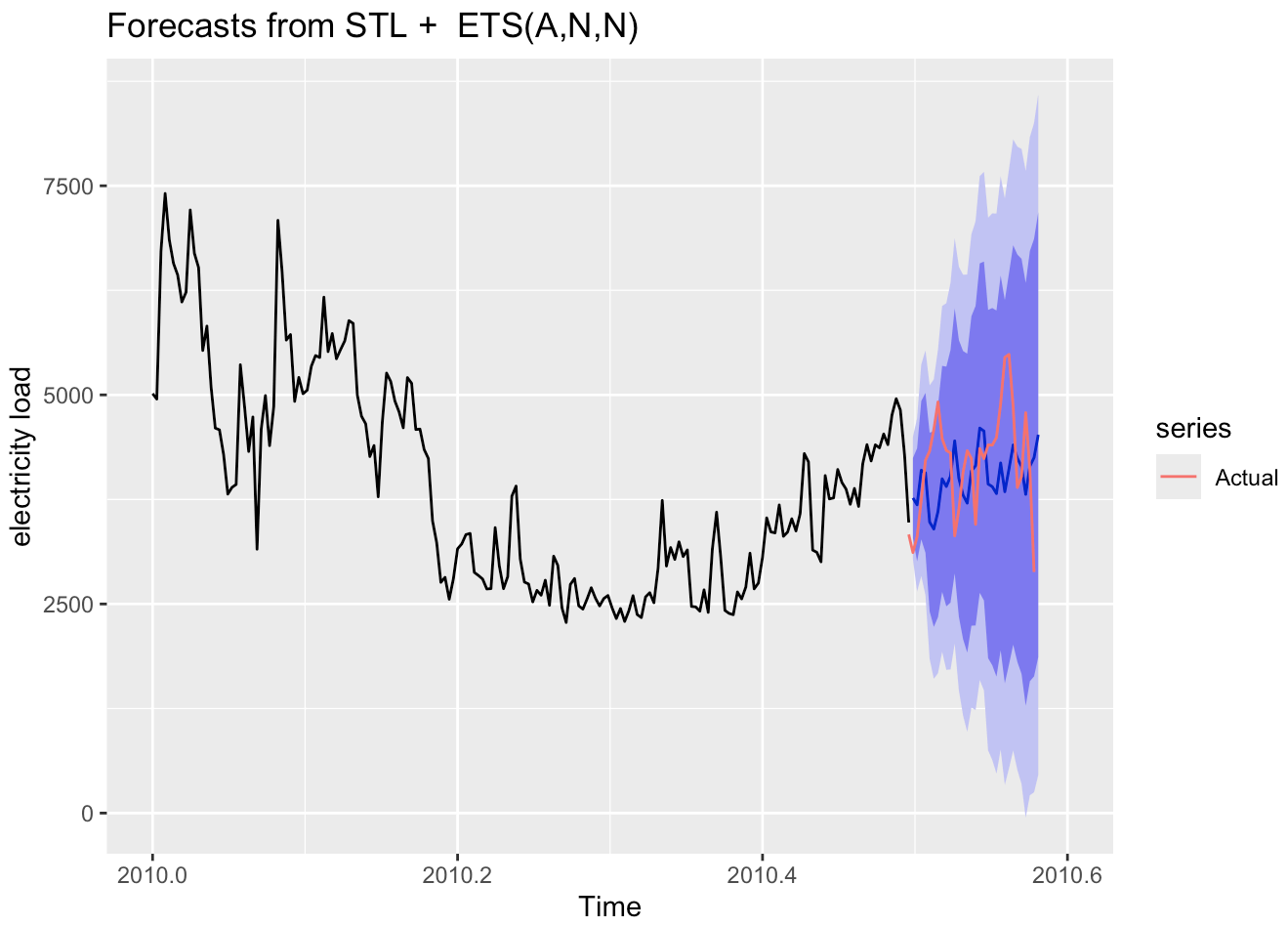
9. Model 5 - STLM

```
fit_stlm <- stlm(train_msts, method = "ets")

fc_stlm <- forecast(fit_stlm, h = length(valid_msts))

autoplot(fc_stlm) +
  autolayer(valid_msts, series = "Actual", PI = FALSE) +
  xlim(2010, 2010.6) +
  ylab("electricity load")
```

```
## Warning in ggplot2::geom_line(ggplot2::aes(x = .data[["timeVal"]], y =
## .data[["seriesVal"]], : Ignoring unknown parameters: `PI`
```

```
## Scale for x is already present.
## Adding another scale for x, which will replace the existing scale.
```

```
## Warning: Removed 1825 rows containing missing values or values outside the scale rang
e
## (`geom_line()`).
```




```
summary(fit_stlm)
```

```
##                Length Class    Mode
## stl            10035  mstl     numeric
## model             19  fc_model list
## modelfunction      1  -none-   function
## lambda             0  -none-   NULL
## x               2007  msts     numeric
## series             1  -none-   character
## m                  1  -none-   numeric
## fitted          2007  ts       numeric
## residuals       2007  ts       numeric
```

```
accuracy(fc_stlm, valid_msts)
```

```
##                      ME     RMSE      MAE        MPE      MAPE      MASE
## Training set   0.6562383 372.4997 274.6701 -0.4702127  7.569894 0.3890829
## Test set     213.8762175 745.5031 618.0582  2.7354526 14.794196 0.8755080
##                    ACF1 Theil's U
## Training set 0.08022665        NA
## Test set     0.46877480  1.337259
```

> Model performance interpretation: Among the five models evaluated, The baseline TBATS model consistently achieved the lowest RMSE, MAE, and MAPE on the test set, indicating best predictive accuracy. In contrast, The other 4 exhibited higher forecast errors, with the ARIMA + Fourier model performing the worst across all metrics. Notably, the Theil's U statistic for Model 1 is closest to 1, suggesting that its performance is competitive relative to a naïve benchmark, while other models significantly unperformed.

Export

Export Function

```
export_submission <- function(forecast_obj, filename) {

  # extract forecast values
  fc_values <- as.numeric(forecast_obj$mean)

  # create July 2011 dates
  dates <- seq(as.Date("2011-07-01"), as.Date("2011-07-31"), by = "day")

  # keep only first 31 forecast values
  df <- data.frame(
    date = dates,
    load = fc_values[1:length(dates)]
  )

  # write csv
  write.csv(df, filename, row.names = FALSE)
}
```

TBATS baseline model

```
export_submission(forecast_tbats, "tbats_submission.csv")
```

SARIMAX model

```
export_submission(fc_sarimax, "SARIMAX_submission.csv")
```

Fourier Terms

```
export_submission(forecast_arima_fourier,"ARIMA_Fourier.csv")
```

MSTL

```
export_submission(fc_mstl,"MSTL_ETS.csv")
```

STLM

```
export_submission(fc_stlm,"STLM.csv")
```